

Transactions JDBC

Auto-commit, commit et rollback

Rappel : Transaction JDBC

Une **transaction JDBC** est un ensemble d'instructions SQL exécutées comme une seule unité logique.

Objectif : garantir la cohérence des données.

Propriétés **ACID** :

- Atomicité : tout ou rien
- Cohérence
- Isolation
- Durabilité

Auto-commit : principe

- Par défaut en JDBC : **autoCommit = true**
- Chaque requête SQL est **validée automatiquement** dès son exécution. Aucune gestion explicite des transactions.

Effet de `setAutoCommit(true)`

- Chaque requête = commit immédiat
- `commit()` inutile
- `rollback()` sans effet

Mode simple mais **dangereux** pour les opérations multiples.

Sans transaction (problème)

- INSERT **inscription OK**
- UPDATE **cours échoue**

→ **Base incohérente**

```
conn.setAutoCommit(true);  
  
stmt.executeUpdate("INSERT INTO inscription ...");  
stmt.executeUpdate("UPDATE cours SET places = places - 1");
```

Si la deuxième requête échoue :

- L'étudiant est inscrit
- Le nombre de places n'est pas mis à jour

Base de données incohérente

Désactiver l'auto-commit

- `setAutoCommit(false)`
- Le développeur contrôle commit / rollback

```
conn.setAutoCommit(false);  
  
stmt.executeUpdate("INSERT INTO inscription ...");  
stmt.executeUpdate("UPDATE cours SET places = places - 1");  
  
conn.commit();
```

Les deux requêtes sont validées **ensemble**

```
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, user, password);
    conn.setAutoCommit(false); // début transaction
    PreparedStatement ps1 = conn.prepareStatement(
        "INSERT INTO inscription(id_etudiant, id_cours) VALUES (?, ?)");
    ps1.setInt(1, idEtudiant);
    ps1.setInt(2, idCours);
    ps1.executeUpdate();
    PreparedStatement ps2 = conn.prepareStatement(
        "UPDATE cours SET places = places - 1 WHERE id_cours = ?");
    ps2.setInt(1, idCours);
    ps2.executeUpdate();
    conn.commit(); // tout est OK
} catch (Exception e) {
    try {
        if (conn != null) conn.rollback(); // annulation
    } catch (SQLException ex) {
        ex.printStackTrace();
    }
} finally {
    try {
        if (conn != null) conn.setAutoCommit(true);
        if (conn != null) conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Transaction correcte

- INSERT inscription
- UPDATE cours
- commit()

→ Validation globale

Gestion des erreurs avec rollback

- Exception → rollback()
- Aucune modification conservée

```
try {  
    conn.setAutoCommit(false);  
  
    // requêtes SQL  
  
    conn.commit();  
} catch (Exception e) {  
    conn.rollback();  
}
```

En cas d'erreur :

- Aucune modification n'est conservée
- Atomicité respectée

Pourquoi `setAutoCommit(true)` à la fin ?

- Bonne pratique
- Évite effets de bord
- Important avec pool de connexions

```
finally {  
    conn.setAutoCommit(true);  
    conn.close();  
}
```

Pièges classiques

- Aucune transaction implémentée
- `setAutoCommit(false)` oublié
- `rollback()` absent dans le catch
- `commit()` placé trop tôt
- Ne pas réactiver l'auto-commit

Conclusion

- Auto-commit = **simplicité** mais **risques**
- Transactions = **cohérence** et **sécurité**
- Indispensable pour les opérations critiques